

Go Architecture & Design Reviewer Handbook

An Evergreen Engineering Specification & Code Review Benchmark

This handbook establishes an **evergreen engineering standard** for designing, auditing, and reviewing Go codebases. Grounded in authoritative Go community benchmarks (Google Style Guide, Uber Style Guide, Effective Go, and Three Dots Labs Clean Architecture specifications), it serves as a precise reference for technical leads, architects, and code reviewers to evaluate structural integrity, maintainability, and idioms.

1 ARCHITECTURE BOUNDARIES & LAYERING

Production Go applications maintain strict separation between business rules, application orchestration, protocol transport, and infrastructure I/O. Dependencies **MUST** flow strictly inward: **Ports & Adapters** → **Application** → **Domain**.

Core Layering Rules:

- **Domain Core (/internal/domain)**: Houses pure business entities, value objects, and domain logic. It has ZERO dependencies on transport frameworks, database drivers, SQL, or ORM tags.
- **Application Layer (/internal/app)**: Coordinates use cases, command/query handlers, and transaction boundaries. It depends solely on pure domain entities and consumer interfaces.
- **Ports Layer (/internal/ports)**: Handles inbound protocol decoding (HTTP handlers, gRPC services, CLI commands) and response formatting. Handlers **MUST NOT** execute database queries or business state updates directly.
- **Adapters Layer (/internal/adapters)**: Implements outbound infrastructure details (PostgreSQL repositories, gRPC clients, Redis caches). Maps database records into domain models upon read, and domain models back into storage schemas upon write.
- **Bounded Feature Domain Packaging**: Group packages by functional feature domain (e.g., /internal/order, /internal/billing) rather than flat technical layers (/controllers, /models) to prevent circular imports (import cycle not allowed).
- **Domain Model Tag Rules**: Structs in /domain **MUST NOT** carry ORM tags (e.g., gorm:"primaryKey" or db:"user_id"). Basic json:"..." tags on domain structs are acceptable only when the public API contract matches the domain model 1:1. If API schemas diverge, explicit presentation DTOs must be placed in the ports layer.

■ ANTI-PATTERN / CODE SMELL: ORM Leakage in Domain Core

Coupling domain entities directly to database schema ORM tags or executing raw SQL queries inside HTTP/gRPC handlers.

```
// ■ BAD: Domain entity polluted with database ORM tags
package domain

type User struct {
    ID    string `gorm:"primaryKey;column:usr_id" // ORM leakage
    Email string `gorm:"uniqueIndex"`
}
```

■ IDIOMATIC PATTERN: Pure Domain with Adapter Mapping

Keep domain structs 100% pure. Define database-specific storage records inside the adapter package and map them explicitly.

```
// ■ IDIOMATIC: Pure domain entity + separate adapter record mapping
package domain
type User struct { ID, Email string }

package adapters
type userRecord struct {
    ID    string `gorm:"primaryKey;column:usr_id"`
    Email string `gorm:"uniqueIndex"`
}
func (r userRecord) toDomain() *domain.User { return &domain.User{ID: r.ID, Email: r.Email} }
```

2 APPROPRIATE USE OF INTERFACES

Go implements implicit structural duck typing. Interfaces should be small, focused abstractions declared at the point of consumption (caller side) rather than pre-emptively exported alongside concrete implementations.

Core Interface Rules:

- **Consumer-Site Declaration:** Declare interfaces in the package that *uses* them (e.g., `/internal/app`). The consumer defines the exact minimal behavioral contract it requires.
- **Narrow & Granular Scope:** Keep interfaces small—ideally 1 to 2 methods (modeled after standard library abstractions like `io.Reader` and `io.Writer`). Single-method interfaces use agent nouns ending in `-er` (e.g., `UserFetcher`, `OrderSaver`).
- **Accept Interfaces, Return Structs:** Constructors and factory functions return concrete struct pointers (`*T`). Functions accept interface parameters (`I`) to enable caller flexibility.
- **No Preemptive Interfaces:** Do NOT declare interfaces on the implementor side 'for mocking' or before multiple implementations exist. Package exports are already exposed boundaries.
- **Lightweight Unit Testing:** Consumer-defined interfaces eliminate heavy external mocking frameworks (like `gomock`). Unit tests instantiate lightweight inline fake/stub structs directly in test files.
- **Compile-Time Compliance Checks:** Use unexported blank identifier assertions (`var _ Interface = (*Type)(nil)`) in implementation packages to catch interface drift at compile time.

■ ANTI-PATTERN / CODE SMELL: Producer Monolithic Interface

Declaring monolithic, 10+ method interfaces in provider packages alongside concrete structs (e.g., ``UserServiceImpl``).

```
// ■ BAD: Producer-side monolithic interface & Java-style Impl naming
package store

type UserServiceInterface interface { // Monolithic interface exported by producer
    GetUser(id string) (*User, error)
    SaveUser(u *User) error
    DeleteUser(id string) error
    // ... 15 other methods
}
type UserServiceImpl struct{} // Anti-pattern 'Impl' suffix
```

■ IDIOMATIC PATTERN: Consumer-Side Narrow Interface & Test Double

Define narrow 1-method interfaces directly inside the consuming package. Tests pass inline fakes.

```
// ■ IDIOMATIC: Consumer-defined 1-method interface + inline test fake
package app // Consumer package

type UserFetcher interface {
    GetUser(ctx context.Context, id string) (*domain.User, error)
}

type Service struct { fetcher UserFetcher }
func NewService(f UserFetcher) *Service { return &Service{fetcher: f} }

// In app_test.go:
type mockFetcher struct{ user *domain.User }
func (m *mockFetcher) GetUser(ctx context.Context, id string) (*domain.User, error) {
    return m.user, nil
}
```

3 DEPENDENCY MANAGEMENT & LIFECYCLE

Components explicitly receive their dependencies through constructor factory functions (NewService). Global mutable state, singletons, and side-effectful init() functions are strictly prohibited.

Core Dependency Rules:

- **Constructor-Driven Injection:** All dependencies (database connections, loggers, downstream clients) MUST be passed as arguments to constructor functions (NewServer(repo, logger)).
- **No Global State or Singletons:** Package-level var database handles or loggers create hidden parameters, cause data races under concurrent testing, and prevent parallel test execution.
- **No Side-Effectful init() Functions:** init() functions must not perform network I/O, open database pools, or mutate global state.
- **Thin main() & Testable run():** Keep main() minimal. Hand off execution immediately to an unexported run(ctx context.Context) error function. This ensures defer statements (e.g., defer db.Close()) execute reliably during teardown before process exit.
- **Functional Options Pattern:** Manage optional component configuration using Functional Options (WithTimeout(d)) or localized option sub-structs rather than passing giant global *Config structs across layers.

■ ANTI-PATTERN / CODE SMELL: Global State & init() I/O

Storing database pools in global package variables initialized via `init()`, and passing monolithic `\*Config` structs everywhere.

```
// ■ BAD: Global mutable state initialized in init()
package store

var DB *sql.DB // Global variable creates race conditions & tight coupling

func init() {
    var err error
    DB, err = sql.Open("postgres", os.Getenv("DB_URL")) // Side-effectful init
    if err != nil { panic(err) }
}
```

■ IDIOMATIC PATTERN: Explicit Constructor Injection & Thin main()

Inject dependencies explicitly via constructors. Execute application logic inside a testable `run()` entrypoint.

```
// ■ IDIOMATIC: Explicit constructor injection & clean run(ctx) lifecycle
func main() {
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()
    if err := run(ctx); err != nil {
        log.Fatalf("startup error: %v", err)
    }
}

func run(ctx context.Context) error {
    db, err := sql.Open("postgres", os.Getenv("DATABASE_URL"))
    if err != nil { return fmt.Errorf("open db: %w", err) }
    defer db.Close() // Guarantees teardown execution upon run() return

    svc := app.NewService(adapters.NewPostgresRepo(db), app.WithTimeout(5*time.Second))
    return svc.Start(ctx)
}
```

4 CONTEXT USAGE & REQUEST SCOPING

The context.Context package manages deadlines, cancellation signals, and request-scoped metadata across API and process boundaries. Proper context discipline prevents resource leaks and hanging database queries.

Core Context Rules:

- **First Argument Position:** Functions performing I/O, database queries, or RPC calls **MUST** accept ctx context.Context as their explicit first parameter (e.g., func Fetch(ctx context.Context, id string)).
- **Dynamic Call-Stack Flow:** Contexts **MUST** flow dynamically down the call stack as pass-by-value arguments. Contexts **MUST NEVER** be stored inside struct fields or long-lived objects.
- **Explicit Operational Budgets:** Enforce timeouts on remote calls using context.WithTimeout or context.WithDeadline. Always execute defer cancel() immediately after creation.
- **Database Query Cancellation:** Pass context directly to QueryContext, ExecContext, and QueryRowContext. When a context times out or client disconnects, the SQL driver cancels the backend database query.
- **Timeout vs Disconnect Disambiguation:** Distinguish context.DeadlineExceeded (downstream bottleneck → HTTP 504/500) from context.Canceled (upstream client disconnect → log warning, degrade gracefully).
- **Request-Scoped Values & Custom Keys:** Restrict context.WithValue strictly to request-scoped metadata (trace IDs, user tokens). Context keys **MUST** be unexported zero-allocation custom types (type traceKey struct{}) to prevent cross-package collisions. Primitive strings **MUST NOT** be used as keys.

■ ANTI-PATTERN / CODE SMELL: Struct Context & String Keys

Storing `context.Context` inside struct fields or using raw strings as keys in `context.WithValue`.

```
// ■ BAD: Storing context in struct and using string context key
type Worker struct {
    ctx context.Context // Never store context inside struct fields!
}

func SetTrace(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, "trace_id", id) // String key causes collisions
}
```

■ IDIOMATIC PATTERN: First-Param Context & Unexported Key Type

Pass context as first parameter. Use unexported custom struct keys for request metadata.

```
// ■ IDIOMATIC: Context as first argument & unexported key type
type traceKey struct{} // Unexported zero-allocation key type

func WithTraceID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, traceKey{}, id)
}

func (s *Service) FetchData(ctx context.Context, id string) (*Data, error) {
    ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel() // Releases context resources promptly
    return s.repo.QueryContext(ctx, id)
}
```

5 ERROR HANDLING & BOUNDARY TRANSLATION

Errors in Go are normal values handled explicitly at the point of occurrence. Standardizing error definitions and translating errors at layer boundaries preserves operational context without leaking infrastructure implementation details.

Core Error Handling Rules:

- **Domain Sentinel Errors:** Core business logic declares pure domain sentinel errors (`var ErrNotFound = errors.New("entity not found")`) or custom error types in the application/domain layer.
- **Boundary Translation:** Low-level errors MUST be translated at layer boundaries:
 - *Adapter Boundary:* Storage adapters catch driver errors (`sql.ErrNoRows`) and translate them to domain sentinels (`domain.ErrNotFound`).
 - *Port Boundary:* Transport handlers catch domain sentinels and map them to protocol status codes (HTTP 404 Not Found, gRPC codes.NotFound). Domain logic MUST NOT return HTTP status codes.
- **Contextual Error Wrapping:** Annotate errors as they ascend the call stack using `fmt.Errorf("...: %w", err)` with the `%w` verb. This preserves the unwrappable error chain for `errors.Is` and `errors.As` inspection.
- **Handle Errors Exactly Once:** Never log an error and return it in the same function. Handle the error once at the outermost boundary (e.g., log in middleware or return to client).
- **No Raw Error Leakage:** Never expose raw database error strings or internal stack traces in HTTP JSON responses.

■ ANTI-PATTERN / CODE SMELL: Raw Error Leakage & Double Handling

Leaking raw `sql.ErrNoRows` or database driver text directly into HTTP responses, or logging and returning the same error.

```
// ■ BAD: HTTP handler executing DB query & leaking raw SQL error
func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    user, err := h.db.Query("SELECT ...") // Handlers shouldn't run SQL!
    if err != nil {
        log.Printf("DB error: %v", err) // Double handling: logged AND returned
        http.Error(w, err.Error(), 500) // Leaking raw sql.ErrNoRows to client!
        return
    }
}
```

■ IDIOMATIC PATTERN: Boundary Translation & %w Wrapping

Translate errors at adapter and port boundaries. Wrap errors contextually with %w.

```
// ■ IDIOMATIC: Boundary translation & contextual wrapping (%w)
// In Adapter: Translate DB error to Domain Sentinel
func (r *PostgresRepo) FindUser(ctx context.Context, id string) (*domain.User, error) {
    var u userRecord
    if err := r.db.GetContext(ctx, &u, "SELECT ...", id); err != nil {
        if errors.Is(err, sql.ErrNoRows) { return nil, domain.ErrNotFound } // Translated
        return nil, fmt.Errorf("querying user %s: %w", id, err) // Context wrapped
    }
    return u.toDomain(), nil
}

// In HTTP Port: Map Domain Sentinel to HTTP Status Code
func (h *HTTPPort) GetUser(w http.ResponseWriter, r *http.Request) {
    user, err := h.app.GetUser(r.Context(), r.PathValue("id"))
    if err != nil {
        if errors.Is(err, domain.ErrNotFound) {
            http.Error(w, `{"error":"user not found"}`, http.StatusNotFound)
            return
        }
        http.Error(w, `{"error":"internal server error"}`, http.StatusInternalServerError)
        return
    }
    json.NewEncoder(w).Encode(user)
}
```

6 CONCURRENCY GOVERNANCE & LIFECYCLE

Concurrency in Go must be explicit and deterministic. A foundational rule dictates that a goroutine **MUST NEVER** be launched without a clear, deterministic strategy for its termination. The caller owns concurrency.

Core Concurrency Rules:

- **Caller Owns Concurrency:** Core domain and application logic **MUST** remain synchronous by default. Leave asynchronous execution decisions to the caller frame.
- **No Fire-and-Forget Goroutines:** Spawning untracked go func() calls inside internal business methods causes memory leaks, unhandled panics, and corrupts graceful shutdown.

- **Goroutine Lifecycle Tracking:** Always track background goroutine execution using `sync.WaitGroup` or `golang.org/x/sync/errgroup`.
- **Structured Lifecycle Management (`errgroup`):** Use `errgroup.WithContext(ctx)` in the application entrypoint to coordinate concurrent HTTP listeners, background queue consumers, and signal handlers.
- **Deterministic Graceful Shutdown:** Upon receiving OS termination signals (`SIGTERM`, `SIGINT`), immediately fail readiness probes, wait for load balancer propagation (e.g., 5s), invoke `http.Server.Shutdown(ctx)` with a bounded timeout, and drain background workers.

■ ANTI-PATTERN / CODE SMELL: Fire-and-Forget Goroutine Leak

Spawning untracked background goroutines inside HTTP handlers or business methods without context cancellation or `WaitGroup` tracking.

```
// ■ BAD: Fire-and-forget goroutine inside business logic
func (s *OrderService) CompleteOrder(ctx context.Context, id string) error {
    // Untracked background goroutine! If app shuts down or panics, work is lost.
    go func() {
        s.emailClient.SendReceipt(id) // Memory leak & race condition risk
    }()
    return s.repo.MarkComplete(ctx, id)
}
```

■ IDIOMATIC PATTERN: Coordinated Shutdown via `errgroup`

Orchestrate asynchronous tasks and graceful shutdown using `errgroup` and `signal.NotifyContext`.

```
// ■ IDIOMATIC: Coordinated goroutines and graceful shutdown with errgroup
func run(ctx context.Context) error {
    ctx, stop := signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)
    defer stop()

    g, gCtx := errgroup.WithContext(ctx)
    srv := &http.Server{Addr: ":8080", Handler: mux}

    // Worker 1: Serve HTTP
    g.Go(func() error {
        if err := srv.ListenAndServe(); err != nil &&& !errors.Is(err, http.ErrServerClosed) {
            return fmt.Errorf("http server: %w", err)
        }
        return nil
    })

    // Worker 2: Graceful Shutdown Monitor
    g.Go(func() error {
        <-gCtx.Done() // Wait for signal cancellation
        shutdownCtx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
        defer cancel()
        return srv.Shutdown(shutdownCtx) // Drains in-flight requests cleanly
    })

    return g.Wait() // Waits for all goroutines to finish
}
```

7 CODE REVIEWER GRADING MATRIX

Use this consolidated quick-reference matrix during code reviews to evaluate pull requests, identify architectural code smells, and mandate idiomatic Go remediations.

Architectural Focus Area	Review Requirement Check	Anti-Pattern / Smell Indicator	Idiomatic Remediation Snippet
1. Domain Boundaries	Are ORM tags or database schemas isolated from domain models?	Structs in /domain containing gorm or db tags.	Define pure domain structs; move ORM tags to /adapters userRecord. type User struct { ID string }
2. Package Layout	Is code packaged by feature domain rather than technical function?	Top-level /controllers or /models folders creating import cycles.	Group packages by business feature domain: package order in internal/order/
3. Interface Scoping	Are interfaces declared on the consumer side and kept small (1-2 methods)?	Provider exporting monolithic UserServiceInterface with 10+ methods.	Declare narrow consumer-side interfaces: type Fetcher interface { GetUser(...) }
4. Interface Factory	Do constructors accept interfaces and return concrete struct pointers?	Constructor returns interface: func New() ServiceInterface.	Accept interfaces, return struct pointers: func NewService(f Fetcher) *Service
5. Dependency Injection	Are dependencies injected explicitly via constructor functions?	Global package variables (var DB *sql.DB) or init() I/O calls.	Inject via explicit constructors: func NewService(r Repo) *Service
6. Entrypoint Lifecycle	Is main() kept thin by delegating to a testable run(ctx) function?	Complex initialization or signal handling written directly inside main().	Delegate to run(ctx) error and defer resource cleanup: defer db.Close() inside run()
7. Context Semantics	Is ctx the 1st parameter and passed dynamically without struct storage?	type Worker struct { ctx context.Context } or raw string context keys.	Pass ctx as 1st arg. Use custom unexported key type: type key struct{}
8. Error Boundaries	Are driver/DB errors translated to domain sentinels at adapter boundaries?	Exposing sql.ErrNoRows or database driver text to HTTP responses.	Map DB error to domain sentinel: if errors.Is(err, sql.ErrNoRows) { return domain.ErrNotFound }
9. Contextual Errors	Are errors wrapped with %w and handled exactly once?	Logging an error and returning it in the same function frame.	Wrap contextually upstream: fmt.Errorf("get user %s: %w", id, err)
10. Concurrency Control	Are background tasks tracked and stay synchronous in business logic?	Firing untracked go func() calls inside internal domain/app logic.	Keep business logic synchronous. Orchestrate lifecycle via errgroup.WithContext(ctx).